

Pathfinding su mappa

Sebastiano Lambertini

1 June 2026

1 Introduzione

Il progetto riguarda il parsing di un file .omap (XML) di una mappa in una griglia di booleani con le zone non attraversabili e le zone percorribili (parsing.cpp). Su questa mappa ci sono anche i simboli della partenza e dell'arrivo. In path.cpp l'algoritmo (ideato da me e spiegato meglio nella sezione successiva) trova tutti i percorsi che abbiano una lunghezza non superiore al 120% del percorso più breve e che siano differenti per il lato da cui aggirano almeno uno degli ostacoli presenti nel percorso.

2 Principali scelte implementative

2.1 Parsing

Il programma serve per analizzare le diverse possibilità di percorso dati una partenza, un arrivo e degli ostacoli da aggirare. Dunque il primo passo è il parsing della mappa attuato nel file parsing.cpp.

Il file è fatto appositamente per leggere file .omap che non sono altro che degli XML che hanno tutta una parte iniziale relativa ai simboli. Ogni file usa numeri diversi per identificare lo stesso simbolo ma all'inizio dei file vengono identificati tutti i simboli e c'è una corrispondenza individuabile tra i numeri utilizzati in ogni specifico file e gli id generici del mondo dell'orienteering. Dunque il programma crea un corrispettivo tra id generici e id specifici.

C'è un vettore di id generici non oltrepassabili, grazie al quale si possono ottenere i numeri specifici utilizzati dal file per gli oggetti non oltrepassabili. Viene utilizzato lo stesso metodo per gli oggetti pieni.

Tra i simboli dell'orienteering c'è un simbolo che è diventato oltrepassabile da pochi anni quindi può capitare che sia considerato come non oltrepassabile in alcune mappe vecchie, perciò nel programma è lasciata la possibilità di scelta all'utente.

Una volta trovati i numeri, si possono cercare gli "object" (solo quelli con id non oltrepassabile) nel file, e si salvano le coordinate. Una volta trovati tutti gli oggetti non oltrepassabili, si trovano il massimo e il minimo in x e in y e si calcolano quindi i limiti della griglia che viene creata con risoluzione fissa di 800. La risoluzione corrisponde al numero di bool della larghezza mentre la lunghezza viene calcolata in base alle proporzioni dei massimi-minimi in x e y.

La griglia parte libera e, prima vengono segnati i bordi usando l'algoritmo di Bresenham, o le curve di Bezier, poi, se il simbolo è pieno, viene riempito orizzontalmente: per ogni riga si parte dal margine e la riga si riempie dopo aver attraversato un numero dispari di volte il bordo fino a

che non si incontra il bordo successivo di quell'oggetto. Si ottiene così la griglia di bool che viene stampata in CSV per essere visualizzata meglio. La griglia, la posizione di partenza e arrivo passano al pathfinding che restituirà i percorsi.

2.2 Pathfinding

Per spiegare l'algoritmo viene prima data un'idea generale e poi qualche funzione viene trattata più nel dettaglio. Il pathfinding parte controllando se si può andare in linea retta verso l'arrivo e quindi controllando se sono liberi tutti i punti da partenza ad arrivo, utilizzando l'algoritmo di Bresenham. Se non sono liberi, l'algoritmo si ferma al primo ostacolo che incontra e trova il punto situato prima del primo ostacolo che chiameremo P_0 . Si divide quindi nel percorso che gira in senso orario e in quello che gira in senso antiorario. Per ognuno dei due rami segue il bordo fino a che non vede spazio in direzione dell'arrivo, ovvero finché il primo bool in direzione non è libero. Si salva quel punto che chiameremo C. Segue il bordo all'indietro finché non vede spazio in direzione della partenza e si salva quel punto che chiameremo P e tutti i punti del bordo tra P e C. Crea il percorso formato da partenza, P, C con tutti i punti della griglia della retta partenza $\rightarrow$ P e tutti i punti del bordo.

A questo punto viene riapplicata la funzione intera con, come partenza, C e, come arrivo, sempre l'arrivo.

Quando l'ultimo C vedrà l'arrivo e non solo spazio in direzione si chiuderà il percorso che verrà pulito: verranno tolti tutti i nodi in eccesso e le deviazioni inutili. E se la retta $P \rightarrow$ partenza o $P \rightarrow C_{precedente}$ non è completamente libera [perché, ricordo, il mio algoritmo passa per P_0 (e non lo salva: il percorso dalla partenza va diretto a P)] viene applicato nuovamente l'algoritmo con, come partenza, la partenza stessa e, come arrivo, il punto prima di quello che non vede punti precedenti, per essere certi di avere tutti i percorsi.

Una volta che si hanno tutti i percorsi si ricalcola la lunghezza del percorso per sicurezza e si filtrano quelli lunghi più del 120% del percorso più corto.

2.2.1 Funzioni particolari

Per seguire il bordo c'è un movimento 8-direzionale con priorità. Quindi l'algoritmo cerca da che parte è l'ostacolo e prova prima ad andare in quella direzione poi, per esempio, se sta seguendo il bordo in senso orario, girerà verso sinistra fino a che non c'è una cella libera in quella direzione. Controlla di avere l'ostacolo vicino e, se vede spazio in direzione dell'arrivo, prosegue.

Per pulire i percorsi parte dall'arrivo e cerca il punto più lontano che vede, quindi partendo dalla partenza e poi avvicinandosi all'arrivo. Se ne vede uno vengono tolti tutti i punti tra i due che si vedono. Se non ne vede nessuno invece, viene applicato nuovamente l'algoritmo come spiegato sopra.

La funzione che pulisce viene applicata finché non si ottiene due volte di fila lo stesso percorso. Questo serve per pulire anche la zona dell'unione tra i paths trovati applicando l'algoritmo tra la partenza e quei punti che non vedono punti prima, e la fine del percorso. Infatti entrambi i tratti sono "puliti" ma può essere tolto qualcosa nell'unione.

3 Come utilizzare il programma

I file sono main.cpp, parsing.cpp, path.cpp, test.cpp e path.hpp e parsing.hpp sono i file header. Serve utilizzare almeno C++ 20. Nel file zip sono anche presenti i file test_su_omap 1,2,3,4 e 5. Da utilizzare per gli ultimi 5 test. Da aggiungere alla build di Cmake se utilizzato

4 Input e output

Ci sono molti test sull'algoritmo con varie configurazioni particolari con l'esportazione in CSV per visualizzarle. Per testarlo con configurazioni a piacere lascio istruzioni qui di seguito.

4.1 Input

Per l'input del programma, se si vuole provare a costruire oggetti di forme a piacere e testare il programma, consiglio il programma gratuito e opensource OpenOrienteeringMapper: <https://www.openorienteeing.org/apps/mapper/>. Altrimenti si può provare a costruire ostacoli, partenza e arrivo su un qualsiasi programma online che dia le coordinate delle posizioni, e costruire gli ostacoli come segue. In questo caso sconsiglio l'utilizzo di curve a meno che non si abbia un programma che utilizza le curve di Bezier cubiche.

Rispettivamente per la partenza e per l'arrivo si utilizzino:

```
< objecttype = "0" symbol = "160" >< coordscount = "1" > 1729163969;< /coords >< /object >
```

(1)

```
< objecttype = "0" symbol = "166" >< coordscount = "1" > 14876-82251;< /coords >< /object >
```

(2)

sostituendo le coordinate con quelle della propria mappa.

Per gli ostacoli consiglio di utilizzare un solo simbolo e solo linee rette costruendolo in questo modo:

```
< objecttype = "1" symbol = "136" >< coordscount = "7" > -12912 - 77230; -10688 - 60714;  
-1319 - 58174; -4019 - 72625; 10274 - 61032; -2113 - 80406; -12912 - 7723018;< /coords >  
< patternrotation = "0" >< coordx = "0" y = "0" / >< /pattern >< /object >
```

modificando il numero di punti e le coordinate stesse. Nel numero di punti se ne indichi uno in più del totale perchè l'ultimo punto è uguale al primo e chiude l'oggetto. Per l'ultima coordinata si tenga il flag finale 18.

Una volta posizionati gli ostacoli, partenza e arrivo, si copi il testo commentato in fondo al file di test e si modifichi la parte finale del file dove compaiono gli oggetti. Ci sarà un commento che dirà da dove iniziare a rimuovere e uno che indica fino a dove farlo ("Modificare gli oggetti qui sotto" e "Modificare/rimuovere fino a qui"). Sostituire i propri oggetti tra i due commenti. Si salvi il file come file txt nella stessa cartella del programma e si avvii il programma.

In parsing.cpp c'è anche l'esportazione in CSV, quindi quella può essere una valida alternativa per visualizzare la disposizione degli ostacoli creati.

4.2 Output

Il programma restituirà, nella stessa cartella del file, un file chiamato *mappa_con_percorso*. Se si è scaricato OpenOrienteeringMapper basta aprire il file con questo programma.

Altrimenti aprire il file come file di testo e, nella stessa zona dei commenti del file di input, ci saranno degli object in più: sono i percorsi con le loro coordinate. Ci sarà sempre un commento ("Qui iniziano i percorsi" e "Qui finiscono i percorsi"). Visualizzare i percorsi sul programma originale utilizzato per l'input.

Non è presente un'esportazione in CSV dei percorsi in quanto buona parte del programma tratta anche la complessa trasformazione da percorsi in una griglia con movimenti solo 4-direzionali a semplificazioni con nodi solo nei punti necessari.

5 Risultati ottenuti

I risultati ottenuti del parsing del file sono ottimi. I risultati ottenuti del pathfinding sono efficaci nella propagazione in avanti. Lo sono anche nella pulizia dei nodi e delle deviazioni all'indietro. Per quanto riguarda la riapplicazione all'indietro di `trova_paths`, ovvero nel caso in cui un nodo non veda nessuno dei precedenti (per esempio se nel tratto $A \rightarrow P$ c'è un ostacolo in mezzo che non c'è tra A e P_0), è efficace se applicata 1 o 2 volte. Se deve essere applicata varie volte per trovare percorsi validi, rischia di esplodere nella ricerca e trovare qualche percorso in più, rispetto a quelli corretti, o addirittura entrare in loop e non terminare la ricerca. Questi percorsi in più sono sia percorsi equivalenti a quelli già trovati, sia percorsi non funzionali. Credo non sia un baco dell'implementazione ma piuttosto un baco logico del mio algoritmo.

6 Test

Sono stati preparati vari test su singole funzioni o su mappe preparate artificialmente e qualche test su mappe vere e proprie con numero di percorsi e lunghezza dei percorsi misurata manualmente. Si è cercato di testare le funzioni in casi base e in casi limite. Nei test che utilizzano le mappe per visualizzare il CSV o il percorso si può sempre eseguire con, come input, i file di testo.

7 Intelligenza Artificiale

Ho fatto uso di Claude, Chat GPT e DeepSeek per:

- l'esportazione in CSV della griglia.
- per l'algoritmo di Bresenham e le curve di Bezier.
- per cercare di fare debugging del pathfinding vero e proprio, in questo caso senza successo (mi hanno solo fatto perdere tantissimo tempo).
- per i test da file interi .omap.
- per il flood fill